



Den299-tanch

2026-06-23

自由研究AIアシスタント 仕様書

項目	内容
アプリ名	自由研究AIアシスタント
対象ユーザー	小学生（イベント当日15名程度が使用予定） ?
イベント	大学構内で行うゼミのITイベント
本番URL	https://jiyukenyu.vercel.app/
リポジトリ	https://github.com/Den299-tanch/jiyukenyu
バージョン	jiyukenyu-app_ver2.1

注：アプリ内のチャットボットを使うと開発者のクレジットが減るため不用意に使わない！！—
ただしー往復0.1円にも満たない+制限を設けてあるので主体的な行動のためなら遠慮する必要はな

1. アプリのフロー解説。（本当に）

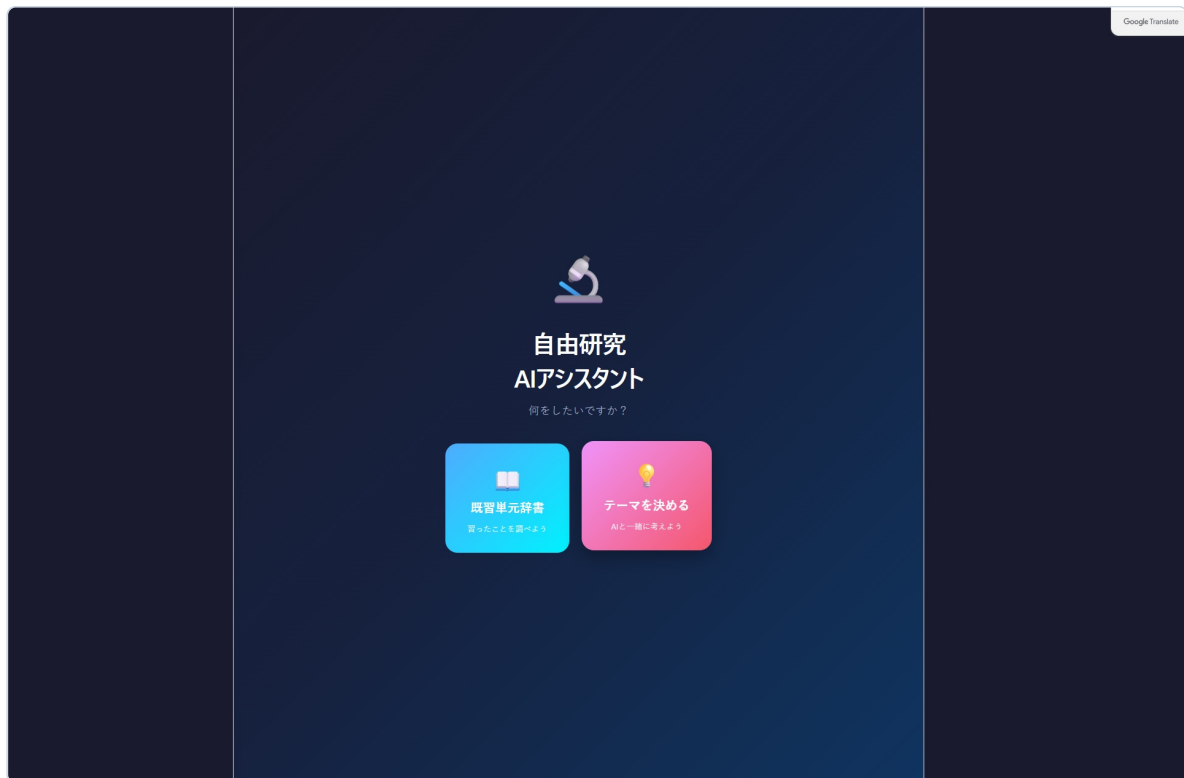
本アプリは「既習単元辞書」と「テーマを決める（AIチャット）」の2つの機能を持つ、小学生向けの自由研究支援ツールである。

画面遷移の流れ



① タイトル画面

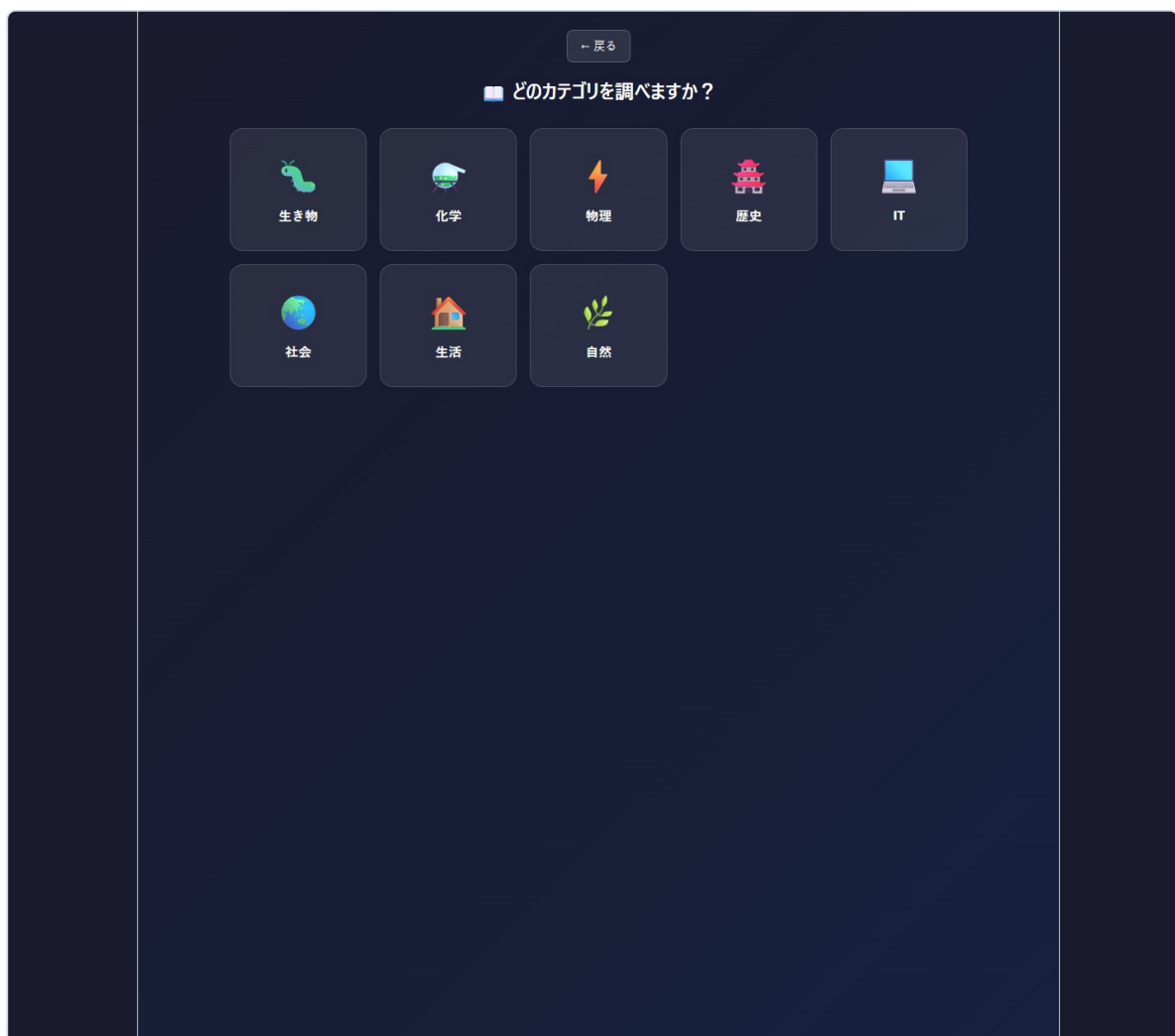
UIについてはあまり自分で設計していないのでかなり適当。



タイトル画面

アプリの入り口。2つの大きなカードボタンで「**既習単元辞書**（習ったことを調べよう）」と「**テーマを決める**（AIと一緒に考えよう）」を選ばせる。背景はダークネイビーで、子どもにも刺さるカラフルなグラデーションでボタンを目立たせている。

② 辞書カテゴリ選択



辞書カテゴリ選択

「既習単元辞書」を選んだ場合に遷移する。8カテゴリ（生き物・化学・物理・歴史・IT・社会・生活・自然）から選択。各カテゴリには絵文字アイコンを配

置し、文字が読みにくい子でも視覚的に選べるようにしている。

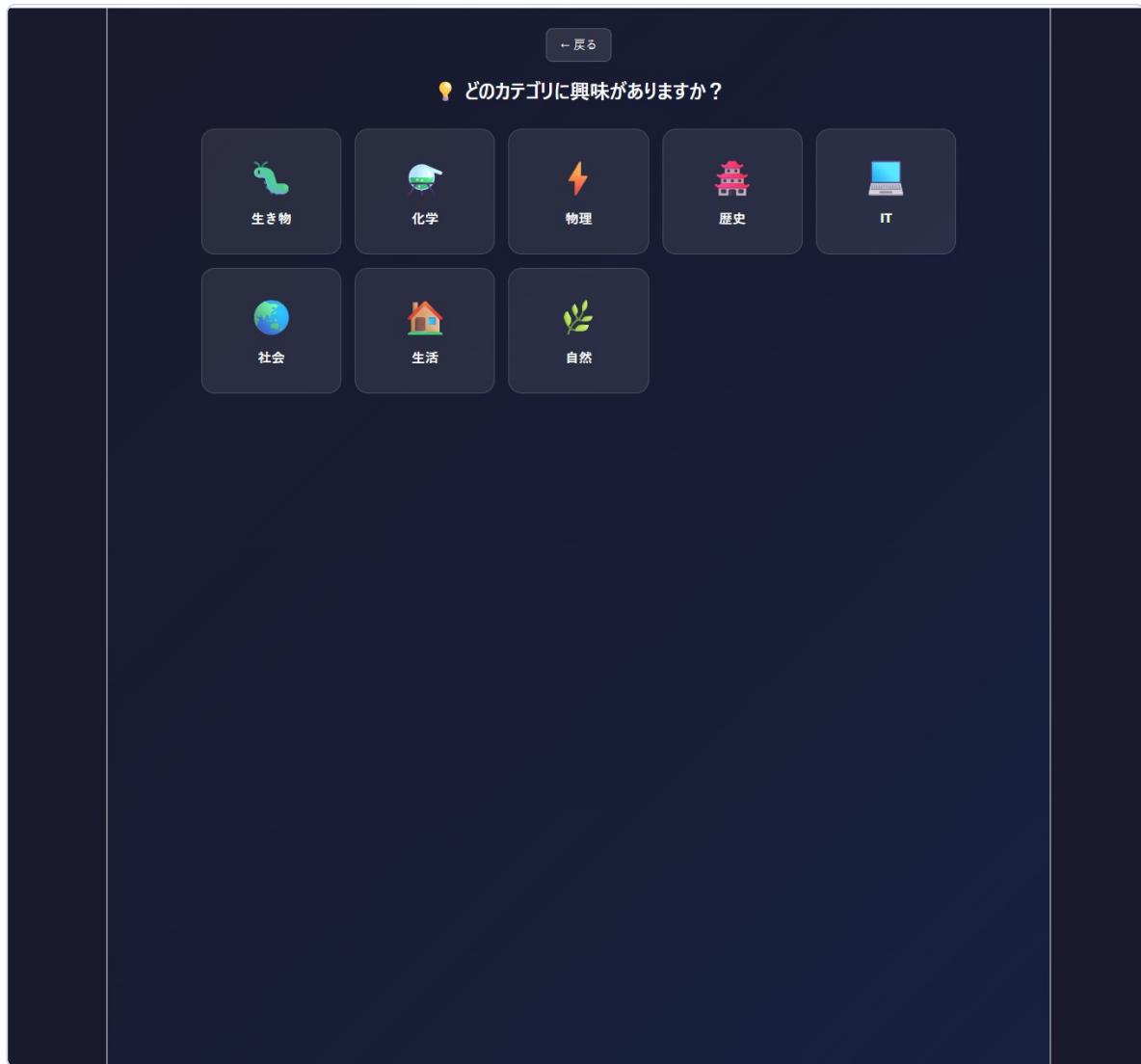
③ 辞書キーワード一覧



辞書キーワード一覧

選択カテゴリに紐づくキーワード（例：生き物→光合成、細胞、食物連鎖）を一覧表示。**クリックで説明文がアコーディオン形式で展開**される。複数同時に開くことも可能。説明文は小学生にも理解できるやさしい言葉で書かれている。

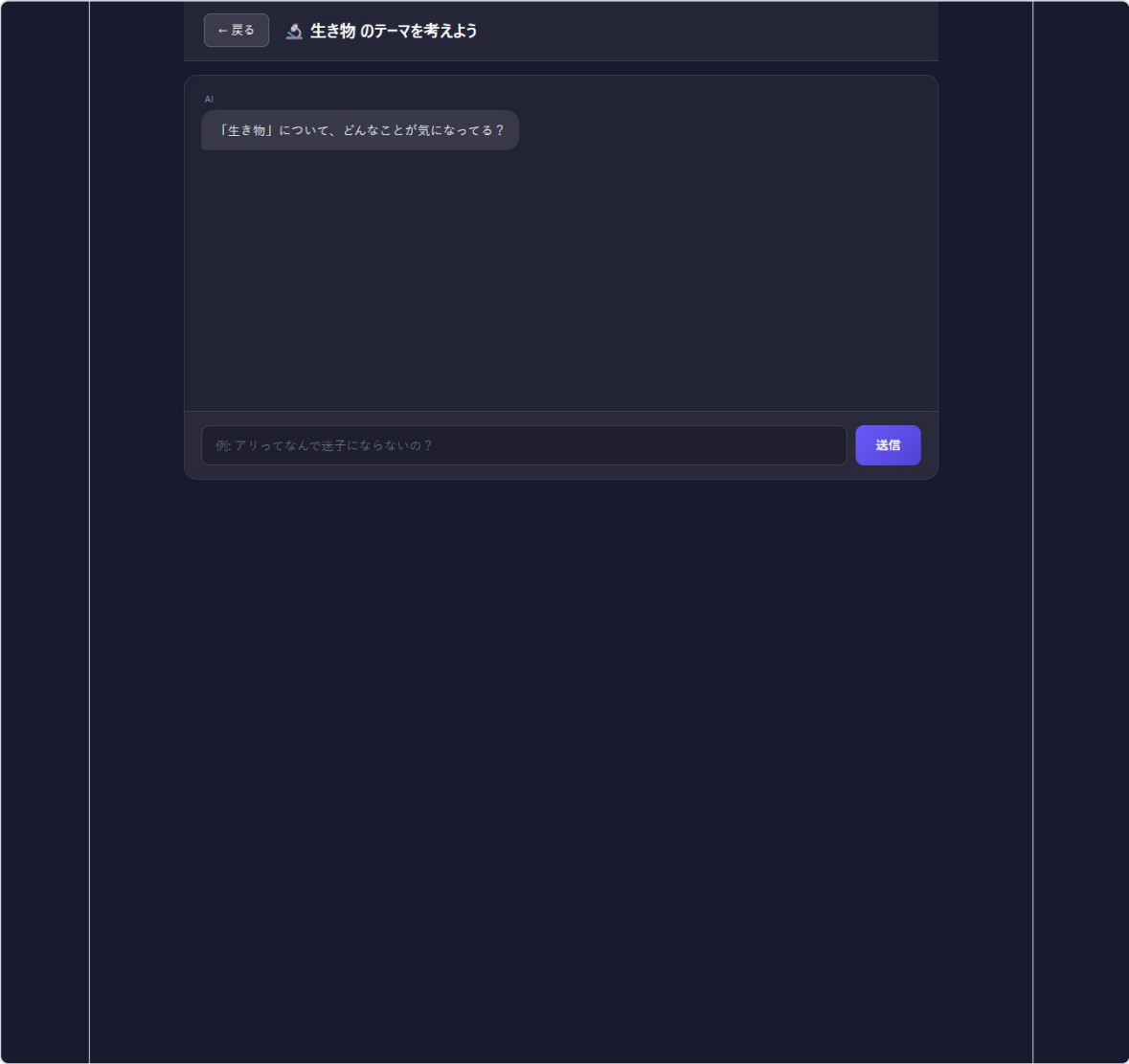
④ テーマ用カテゴリ選択



テーマ用カテゴリ選択

「テーマを決める」を選んだ場合に遷移する。辞書と同じ8カテゴリを共通のコンポーネント（ `CategorySelect.jsx` ）で使い回している。ここで選んだカテゴリが、後続のAIチャットでのシステムプロンプトに反映される。

⑤ AIチャット画面



AIチャット画面

Claude API (sonnet-4-6) と対話しながら自由研究のテーマを考える画面。AIは「答えを直接教える」のではなく、「**なぜだと思う？**」「**試してみたらどうなるかな？**」と問いかけて、**子ども自身に考えさせる先生役**として動作する。入力欄にはプレースホルダで質問例（例：アリってなんで迷子にならないの？）が表示され、最初の一手のハードルを下げている。

SPA URLの変更はしない画面遷移方式

HMR React + Viteの一番の導入理由、リロードなしで即時反映

LLM 大規模言語モデル、今回のClaudeやchat gptなどの文章生成AI

2. 技術スタック 必要十分なツールを使用しているはず

レイヤー	技術	役割
フロントエンド	Vite + React	SPA構築。Viteの高速HMRで開発効率を担保
バックエンド	Node.js + Express	Claude APIへの中継サーバー。APIキーをクライアントに露出させないため必須
フロントホスティング	Vercel（無料プラン）	静的アセット配信 + 自動デプロイ
バックエンドホスティング	Render（無料プラン）	Expressサーバーのホスティング
LLM	Claude API (claude-sonnet-4-6)	対話部分の生成エンジン

デプロイ 自分のPCで動いているアプリをインターネット上（サービス）のサーバーにおいて世界中でアクセスできるようにすること

3. インフラ・コスト構成

開発環境（ローカル）

API （自分のアプリから見て）外部機能を使うための窓口

開発時はフロントとバックの2本立てサーバーで動作させていた：

ポート	役割	起動コマンド
5173	Viteのフロント用開発サーバー	npm run dev
3001	Expressのバックエンドサーバー	node server/index.js

`5173` はViteのデフォルト開発ポート。Reactコンポーネントを編集すると即座にブラウザに反映される（HMR）。`3001` はExpressサーバーで、フロントから `/api/chat` に来たリクエストを受けてClaude APIに中継する。CORSは `cors` パッケージで全許可にしている。

ローカル動作時は `VITE_API_URL` 環境変数を `http://localhost:3001` に設定することで、フロントが正しくローカルのExpressを向く。

本番環境（クラウド）

サービス	プラン	役割	備考
Vercel	無料	フロント（Reactビルド成果物）配信	自分のgithub連携ができていますのでpushで自動デプロイ、つまり更新ができる <code>git push</code> で自動デプロイ。スマホブラウザでの動作確認済み
Render	無料	Expressサーバー稼働	環境変数 <code>CLAUDE_API_KEY</code> をRender側のSecretsに登録
Claude API	従量課金	Anthropic公式	現状すべて開発者個人のクレジットで稼働

Renderの無料プランの注意点（スリープ問題）

Renderの無料プランは、**15分間アクセスがないとサーバーが自動的にスリープ状態**になる。スリープから復帰する最初の1リクエストは応答までに数十秒（30秒〜1分程度）かかる。

イベント当日の対策案：

- 開場直前にあらかじめ1回チャットを送信して、サーバーをウォームアップしておく
- 定期的にダミーリクエストを送る仕組み（cron-job等）を入れる
- 心配なら有料プランへの一時的なアップグレード（月7ドル程度）

Claude APIのコスト感

低haiku sonnet opusと選べはする

1回目なら約0.03円

`claude-sonnet-4-6` は従量課金（入力・出力トークン量に応じて課金）。今のところ会話1往復あたりおよそ ~~0.05ドル前後（約7円）~~ で推移している。イベント想定15名 × 数往復であれば、**全体でも数百円程度に収まる見込み**。**チャットボットのみならかかっても1,000円程度なはず**

4. 環境変数一覧

もしかしたら難しいことをすればAPIキーを覗けるのかもしれないが、絶対に外部に漏らさない。セキュリティ面で大きな不安があるし、Claudeは特に再設定が面倒くさいため。

変数名	設定場所	用途
<code>CLAUDE_API_KEY</code>	サーバー側 <code>.env</code> （Renderの場合はSecrets）	Anthropic APIの認証キー。 絶対にフロントに置かないこと
<code>VITE_API_URL</code>	フロント側 <code>.env</code> （Vercelの場合はEnvironment Variables）	Expressサーバーのベース URL。ローカル時は <code>http://localhost:3001</code> 、本番時はRenderの URL

注意： `VITE_` で始まる変数はビルド時にバンドルに埋め込まれるため、フロント側のJSファイルから内容が見えてしまう。秘密の値は絶対にここに入れないこと。

5. ディレクトリ構成

```

jiyukenyu-app_ver2.1/
├─ index.html                # Viteのエントリ HTML
├─ package.json              # 依存関係・スクリプト定義
├─ vite.config.js            # Viteの設定
├─ eslint.config.js          # ESLint設定
├─ README.md
├─ public/                   # 静的アセット（faviconなど）
├─ src/                       # フロントエンド本体
│  ├─ main.jsx               # Reactのエントリーポイント
│  ├─ App.jsx                # 画面遷移・状態管理の中心
│  ├─ App.css                # 全体スタイル
│  ├─ index.css
│  └─ components/
│     ├─ TitleScreen.jsx     # タイトル画面
│     ├─ CategorySelect.jsx  # カテゴリ選択（辞書/チャット共用）
│     ├─ DictScreen.jsx      # 辞書画面+辞書データ
│     ├─ ChatBox.jsx         # チャット枠
│     ├─ MessageList.jsx     # メッセージ一覧
│     ├─ Message.jsx         # メッセージ1件
│     └─ InputForm.jsx       # 入力フォーム
│  └─ services/
│     └─ Claudeapi.js        # サーバー（/api/chat）への通信ラッパ
└─ server/                   # バックエンド
   └─ index.js               # Express本体+Claude API中継処理

```

6. セットアップ手順

当然開発者のPCにはnode.jsは必要だが、今回はもうすでにデプロイされているためいらないという意味。

本アプリはすでにVercel／Renderにデプロイ済みのため、利用者側でローカル環境を構築する必要はない。ブラウザで本番URL（<https://jiyukenyu.vercel.app/>）を開けばそのまま使える。Node.jsのインストールも不要で、PC・スマホ双方の主要ブラウザから動作する。

~~開発・改修目的でローカルで動かす場合のみ、リポジトリをZIPでダウンロードしてVS Code等で開き、npm install 後に npm run dev（フロント）と node server/index.js（バックエンド）の2本立てで起動する流れになる。~~

7. 軽いコード解説 API部分

src/App.jsx

アプリ全体の状態管理と画面切り替えを担う中心ファイル。screen というstateで現在表示する画面（title / dict-category / chat-category / chat / dict）を制御している。ReactのフックはuseState のみで、ライブラリに依存しないシンプルな構成。

src/services/Claudeapi.js

フロント

フロントからExpressサーバーの /api/chat を叩くラッパー関数 callClaude を提供する。会話履歴（history）とカテゴリ（mode）を fetch でPOSTし、返ってきた応答テキストを取り出す薄い層。

server/index.js

バック

Expressの本体。/api/chat エンドポイントを1つだけ持ち、フロントからのリクエストを受けてAnthropic APIへ中継する。カテゴリ別のシステムプロンプト（PROMPTS）と共通の先生キャラ設定（BASE_SYSTEM）を組み合わせでClaudeに渡している。APIキーは process.env.CLAUDE_API_KEY から取得するため、ソースコードには一切含まれない。

src/components/DictScreen.jsx

辞書画面。DICT_DATA というオブジェクトに各カテゴリのキーワードと説明文がベタ書きされており、これを編集するだけで辞書の中身を簡単に追加・変更できる。~~辞書データの増強はこのファイルを開いて配列に追加するだけで済むので、メンバーでも比較的着手しやすい。~~

データの流れ（再掲）

コードの理解があれば追加や変更は容易ではある
これはプロンプトも辞書も両方とも

```
ブラウザ
  ↓
  fetch('/api/chat', {history, mode})
Claudeapi.js (callClaude)
  ↓
Render上のExpress (/api/chat)
  ↓
  fetch('https://api.anthropic.com/v1/messages')
Anthropic Claude API
  ↓
  応答
Express → フロントへ返却
  ↓
画面にメッセージ追加
```

8. GitHubリポジトリ

- URL：<https://github.com/Den299-tanch/jiyukenkyu>
- メインブランチ：`main`
- 本仕様書の対象：`jiyukenkyu-app_ver2.1` フォルダ配下

publicなので壊そうと思えば壊せるが、zipバックアップがあるのでまあ安心

リポジトリにはバージョン違いの実装フォルダが複数含まれており、本番デプロイは `ver2.1` 系を使用している。Vercel・Renderともに `main` ブランチへのpushで自動デプロイされる設定。

9. 今後の展望

データベース導入

現状は会話履歴・辞書データ・ユーザー情報をすべて**サーバーメモリまたは静的データ**で扱っており、ページをリロードすると会話履歴は消える。これを永続化するためにDBの導入を検討中。

ツール	役割
PostgreSQL	リレーショナルDB本体。ユーザー、会話履歴、辞書を構造化して保存
DBeaver	DB管理用のGUIクライアント。スキーマ確認・データ閲覧・SQL実行が容易

DBを入れることで、後述のユーザーID振り分けや、辞書データのアプリ外編集が現実的になる。

ユーザーID振り分け機構

複数人が同時に使う想定のため、ユーザーを識別する仕組みを導入したい。

- 端末ごとに一意のID（UUID）を割り当て、LocalStorageに保存
- もしくはイベント当日に番号札を配って入力させる方式
- 会話履歴やテーマ案を**ユーザー単位で保存**できるようにする

これにより、当日小学生が「自分が考えたテーマをあとから見返す」運用が可能になる。

その他構想

- ~~会話履歴のエクスポート（PDF/テキスト）~~
- ~~辞書データの管理画面（CSV読み込みやWeb編集）~~
- ~~簡易ログイン（保護者用・先生用）~~

10. 既知の課題・TODO

カテゴリ	内容	優先度
UI/UX改善	全体の見た目の磨き込み、入力中インジケータ、画面遷移アニメーション等	中
辞書の充実	現状各カテゴリ1〜3件しかない。メンバーから集めて20件以上に増やしたい（雛形はあるので追記は容易）	高
CSS／デザイン基盤	Bootstrap・Tailwind等の導入で見た目を底上げ	中

他：重要

プロンプト改善	他：重要	辞書／チャット双方のシステムプロンプトはまだ叩き台。雛形があるので差し替えは容易	中
DBリテラシー		チーム全体でPostgreSQL・SQL・ORMの基礎理解が必要	高（DB導入前提なら必須）
多端末動作テスト	重要	iPhone／Android／タブレット／PCで広く動作確認したい。Vercel側ではスマホ表示確認済	高
当日の保守運用		Renderスリープ問題 、APIキー残量、誰がトラブル対応するか、の役割分担	高（当日直前）
会話履歴の永続化	自：重要	リロードで消える問題（DB導入で同時解決）	中
エラーハンドリング		API失敗時の表示や再試行のUI整備	低

フロー後半部 超重要！！ これがないなら作れない

あくまでここまですべて自分の考え
かなりできているようでまだまだな段階

- ・辞書のカテゴリやキーワードはかなり適当な一例のため変更したい
- ・設定プロンプトはベースのものとそれぞれカテゴリごとに設定できる。現状はかなり適当。
- ・コーディングはClaudeが主に行っているが、色々考慮するとすぐできるものではないので注意

ぶっちゃけ現状なら直接Claude使えばいいだろう
っていう問題がある。アプリとしての価値創造が足りない